Synthetic Dataset Generation for Rare Medicinal Plants

Technical Documentation - Methods, Algorithms & Pipeline

Table of Contents

- 1. [Project Overview](#1-project-overview)
- 2. [Dataset - Species & Sources](#2-dataset--species--sources)
- 3. [Phase 1 - Seed Data Collection](#3-phase-1--seed-data-collection)
- 4. [Phase 2 - Lighting Augmentation (Pix2Pix cGAN)](#4-phase-2--lighting-augmentation-pix2pix-cgan)
- 5. [Phase 3 - Background Harmonization GAN](#5-phase-3--background-harmonization-gan)
- 6. [Phase 4 - CycleGAN (Unpaired Domain Translation)](#6-phase-4--cyclegan-unpaired-domain-translation)
- 7. [Phase 5 - Weather Scene Simulation](#7-phase-5--weather-scene-simulation)
- 8. [Final Dataset Summary](#8-final-dataset-summary)
- 9. [Tools & Environment](#9-tools--environment)

1. Project Overview

Goal: Build a large, annotated synthetic image dataset of rare medicinal plants to support downstream machine learning classification tasks, where real-world data is extremely limited due to the endangered/rare nature of these species.

Core Problem: Rare medicinal plants have very few publicly available images - often fewer than 30 research-grade photographs exist per species on any scientific platform. This is insufficient to train a robust classifier. Standard data augmentation (flips, crops, rotations) introduces only marginal diversity. We therefore use a multi-stage GAN pipeline to synthesize photorealistic variants covering:

- Different lighting conditions (bright sunlight, dim canopy, dark flash photography)
- Different weather environments (summer, winter, monsoon)
- Different imaging contexts (field photography, herbarium specimens)

Platform: Google Colab (T4 GPU)

Framework: PyTorch 2.x

Language: Python 3.13

2. Dataset - Species & Sources

All 12 species are IUCN-listed rare or threatened medicinal plants sourced exclusively from authenticated open-science repositories.

#	Scientific Name	Common Name	IUCN Status	Family
1	Rauvolfia serpentina	Sarpagandha	Endangered	Apocynaceae
2	Nardostachys jatamansi	Jatamansi	Endangered	Caprifoliaceae
3	Taxus wallichiana	Himalayan Yew	Endangered	Taxaceae
4	Aconitum heterophyllum	Ativisha	Endangered	Ranunculaceae
5	Swertia chirayita	Chirayita	Vulnerable	Gentianaceae
6	Commiphora wightii	Guggul	Endangered	Burseraceae
7	Gymnema sylvestre	Gurmar	Least Concern	Apocynaceae
8	Pterocarpus marsupium	Indian Kino	Vulnerable	Fabaceae
9	Coscinium fenestratum	False Calisaya	Endangered	Menispermaceae

10	Dioscorea deltoidea	Wild Yam	Vulnerable	Dioscoreaceae
11	Aquilaria malaccensis	Agarwood	Critically Endangered	Thymelaeaceae
12	Gloriosa superba	Flame Lily	Vulnerable	Colchicaceae

3. Phase 1 - Seed Data Collection

3.1 Data Sources

Two authenticated scientific biodiversity repositories were used:

iNaturalist (api.inaturalist.org/v1)

- Filter: quality_grade=research - only community-verified, expert-confirmed observations
- Metadata available: observer, GPS coordinates, date, license
- License filter: cc-by, cc-by-nc, cc0 only (open for academic use)
- Target: ~15 images per species

GBIF (api.gbif.org/v1)

- Source: Digitized herbarium voucher specimens from global natural history museums
- Filter: mediaType=StillImage, hasCoordinate=true
- Includes: pressed specimens, living plant photographs, botanical illustrations
- Target: ~13 images per species

3.2 Download Algorithm

```
FOR each species in SPECIES_DB:
    query iNaturalist API with taxon_name, quality_grade=research
    download up to 15 images with open licenses

    query GBIF API with species name
    download up to 13 images with media URLs

    save to /raw_images/<species_folder>/
    log to metadata CSV (species, source, URL, license, plant_parts)

OUTPUT: ~28 images per species x 12 species = ~336 raw seed images
```

3.3 Metadata Schema

Each downloaded image is logged with:

Field	Description
‘species_name‘	Scientific name (folder name)
‘common_name‘	Common English name
‘source‘	‘iNaturalist‘ or ‘GBIF‘
‘image_url‘	Original URL
‘license‘	Copyright license
‘plant_parts‘	Leaf, flower, fruit, bark, root (where known)
‘active_compounds‘	Key medicinal compounds for the species
‘image_path‘	Local saved path

4. Phase 2 - Lighting Augmentation (Pix2Pix cGAN)

4.1 Motivation

Real-world plant photographs are taken under highly variable lighting conditions:

- Bright: Direct sunlight, solar glare, washed-out highlights
- Dim: Canopy shadow, overcast sky, underexposed mobile photos
- Dark: Night photography with mobile flash, high ISO noise

A classifier trained only on iNaturalist images (mostly well-lit) will fail on field images taken under poor conditions. We simulate all three conditions both via a physics-based photometric engine (for training data) and via a Pix2Pix GAN (for learning the full image distribution).

4.2 Photometric Engine (Rule-Based)

Before GAN training, we generate paired training data using a differentiable photometric simulation:

Bright condition:

```
brightness x 1.6
contrast x 0.85
blend with warm overlay (255, 255, 220) at alpha = 0.18
```

Dim condition:

```
brightness x 0.55
contrast x 1.10
saturation x 0.75
```

Dark condition:

```
brightness x 0.28
contrast x 1.20
add Gaussian noise N(0, sigma=8) to all channels
```

4.3 Pix2Pix Architecture

Generator: U-Net with Skip Connections

```
Input: 6 channels (3 RGB image + 3 condition map broadcast)

Encoder (downsampling path):
  Conv2d(6->64, k=4, s=2) + LeakyReLU(0.2)
  Conv2d(64->128, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(128->256, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(256->512, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(512->512, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)  [bottleneck]

Decoder (upsampling path with skip connections):
  ConvTranspose2d(512->512) + InstanceNorm + ReLU + Dropout(0.5)
  ConvTranspose2d(1024->256) + InstanceNorm + ReLU           [concat with encoder]
  ConvTranspose2d(512->128)  + InstanceNorm + ReLU
  ConvTranspose2d(256->64)   + InstanceNorm + ReLU
  ConvTranspose2d(128->3, k=4, s=2) + Tanh                  [output RGB]
```

> *Note: All ReLU/LeakyReLU activations use ‘inplace=False‘ to prevent autograd errors through skip connections.*

Discriminator: 70x70 Markovian PatchGAN

```
Input: 6 channels (real or fake image concatenated with condition)

Conv2d(6->64,   k=4, s=2) + LeakyReLU(0.2)
Conv2d(64->128, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
Conv2d(128->256,k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
Conv2d(256->512,k=4, s=1) + InstanceNorm + LeakyReLU(0.2)
Conv2d(512->1,  k=4, s=1) + Sigmoid                        [patch-level real/fake]
```

The PatchGAN outputs a 30x30 grid of real/fake predictions. Each patch covers a 70x70 receptive field in the input. This enforces local texture realism rather than global coherence.

4.4 Loss Function

$$\mathcal{L}_{total} = \mathcal{L}_{GAN} + \lambda \cdot \mathcal{L}_{L1}$$

$$\mathcal{L}_{GAN} = \mathbb{E}[\log D(x, y)] + \mathbb{E}[\log(1 - D(x, G(x, c)))]$$

$$\mathcal{L}_{L1} = \mathbb{E}[\|y - G(x, c)\|_1]$$

Where:

- x = input image, y = target image, c = lighting condition vector
- $\lambda = 100$ (L1 weight - preserves plant anatomy)
- Loss: BCEWithLogitsLoss for GAN term

4.5 Training Configuration

Parameter	Value
Image size	256 x 256
Batch size	4
Epochs	100
Optimizer	Adam (beta =0.5, beta =0.999)
LR (Generator)	2x10
LR (Discriminator)	2x10
lambda L1 weight	100

Saved weights: sarpagandha_generator.pth, sarpagandha_discriminator.pth

5. Phase 3 - Background Harmonization GAN

5.1 Motivation

Plant images from iNaturalist feature diverse natural backgrounds (forest floor, rocky terrain, grass). A classifier should be invariant to background and focus only on plant morphology. We trained a GAN to composite segmented plant parts onto controlled backgrounds.

5.2 Architecture

HarmonizationGenerator (U-Net, 7 input channels):

```
Input: foreground (3ch) + background (3ch) + lighting_index (1ch) = 7ch
-> U-Net encoder-decoder with skip connections
-> Output: harmonized composite (3ch, Tanh)
```

BackgroundPatchGAN (Discriminator):

```
Input: composite image (3ch)
-> 4x strided convolutions with InstanceNorm
-> Output: patch-level real/fake map
```

Backgrounds used: Soil (RGB: 101,67,33), Jute sack (182,155,76), Wooden board (120,81,45)

Segmentation method: OpenCV GrabCut (rect-initialized, 5 iterations)

> *Limitation noted: GrabCut is unreliable when plant and background share similar color distributions (green plants on green backgrounds). This was identified as a key technical bottleneck. Production-grade segmentation would require a deep learning model (e.g. Segment Anything Model - SAM).*

Saved weights: sarpagandha_background_gan.pth

6. Phase 4 - CycleGAN (Unpaired Domain Translation)

6.1 Motivation

Pix2Pix requires perfectly paired images (same plant, two lighting conditions). CycleGAN removes this constraint - it learns to translate between two image domains without any paired examples. This is critical here because we cannot photograph the same rare plant twice under different conditions.

- Domain A: Clean, well-lit iNaturalist field photographs
- Domain B: Physics-simulated non-ideal variants (bright/dim/dark photometric engine output)

6.2 Architecture

Two generators (9-block ResNet):

```
CycleGenerator G_AB: Domain A -> Domain B  (clean -> non-ideal)
CycleGenerator G_BA: Domain B -> Domain A  (non-ideal -> clean)

Architecture per generator:
  Input Conv: Conv2d(3->64, k=7) + InstanceNorm + ReLU

  Downsampling:
    Conv2d(64->128,  k=3, s=2) + InstanceNorm + ReLU
    Conv2d(128->256, k=3, s=2) + InstanceNorm + ReLU

  9x ResNet Blocks:
    [Conv -> InstanceNorm -> ReLU -> Conv -> InstanceNorm] + residual skip
    (all activations inplace=False)

  Upsampling:
    ConvTranspose2d(256->128, k=3, s=2) + InstanceNorm + ReLU
    ConvTranspose2d(128->64,  k=3, s=2) + InstanceNorm + ReLU

  Output Conv: Conv2d(64->3, k=7) + Tanh
```

Two discriminators (PatchGAN):

```
CycleDiscriminator D_A: distinguishes real Domain A from G_BA(B)
CycleDiscriminator D_B: distinguishes real Domain B from G_AB(A)

Architecture:
  Conv2d(3->64,    k=4, s=2) + LeakyReLU(0.2)
  Conv2d(64->128,  k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(128->256, k=4, s=2) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(256->512, k=4, s=1) + InstanceNorm + LeakyReLU(0.2)
  Conv2d(512->1,   k=4, s=1)   [patch output]
```

6.3 Loss Functions

Adversarial Loss (LSGAN - Mean Squared Error):

$$\mathcal{L}_{GAN}(G, D_B, A, B) = \mathbb{E}_b[(D_B(b) - 1)^2] + \mathbb{E}_a[D_B(G_{AB}(a))^2]$$

> *LSGAN (MSE) is used instead of BCE - provides smoother gradients and avoids vanishing gradients common with BCE in early training.*

Cycle Consistency Loss:

$$\mathcal{L}_{cyc}(G_{AB}, G_{BA}) = \mathbb{E}_a[\|G_{BA}(G_{AB}(a)) - a\|_1] + \mathbb{E}_b[\|$$

This ensures that translating A->B->A recovers the original image, preventing mode collapse and preserving plant identity.

Identity Loss:

$$\mathcal{L}_{id} = \mathbb{E}_b[\|G_{AB}(b) - b\|_1] + \mathbb{E}_a[\|G_{BA}(a) - a\|_1]$$

Ensures that if a Domain B image is fed to G_AB, it returns unchanged. Preserves color coherence.

Total Generator Loss:

$$\mathcal{L}_{total} = \mathcal{L}_{GAN}^{AB} + \mathcal{L}_{GAN}^{BA} + \lambda_{cyc} \cdot \mat$$

Loss weight	Value
λ_{cyc}	10
λ_{id}	5

6.4 Training Stabilization Techniques

Image Replay Buffer:

```
class ImageBuffer:
    """Stores last 50 generated images.
    Discriminator is trained on a random mix of
    newly generated + previously generated images.
    Prevents discriminator from over-fitting to
    the most recent generator outputs."""
    buffer_size = 50
```

Learning Rate Scheduler:

```
Epochs 1-15:  constant LR = 2x10
Epochs 16-30: linear decay -> 0
```

This allows initial stable learning before gradually reducing updates as the model converges.

6.5 Training Configuration

Parameter	Value
Image size	256 x 256
Batch size	1
Epochs	30
Optimizer	Adam (beta =0.5, beta =0.999)
LR	2x10
Normalization	InstanceNorm (not BatchNorm)
Replay buffer size	50

> *Why InstanceNorm over BatchNorm? InstanceNorm normalizes per-image, per-channel - essential for style transfer tasks where batch statistics would corrupt style information across different plant species.*

Saved weights: G_AB_clean_to_nonideal.pth, G_BA_nonideal_to_clean.pth

7. Phase 5 - Weather Scene Simulation

7.1 Motivation

Medicinal plant images are collected across different seasons and geographies. A robust classifier must recognize the same plant in summer heat, winter cold, and monsoon rain. We simulate three primary seasons via photometric and physical modelling.

7.2 Season Algorithms

Summer

```
Simulates: Strong direct sunlight, heat haze, warm color temperature

1. Brightness x1.5, Contrast x0.9, Saturation x1.2
2. Boost red channel x1.08 (warm color temp)
3. Reduce blue channel x0.88
4. Add Gaussian heat shimmer noise N(0, sigma=5) over full image
```

Winter

```
Simulates: Overcast cold sky, frost fog, desaturated scene

1. Brightness x0.85, Saturation x0.55
2. Boost blue channel x1.15 (cool color temp)
3. Reduce red channel x0.90
4. Blend full image with white fog overlay (RGB: 210,210,210) at alpha=0.22
5. Bottom 25% replaced with icy blue-white snow drift (gradient blend)
6. 250 random snowflake circles overlaid (r=1-4px, alpha=0.5-0.95)
```

Monsoon

```
Simulates: Heavy rain, wet foliage, overcast sky, high humidity

1. Brightness x0.65, Saturation x1.35, Contrast x1.15
2. Boost green channel x1.10 (lush wet foliage)
3. Boost blue channel x1.10 (rain atmosphere)
4. Rain streaks: 120 vertical line segments
   - Random x, random y_start, length 8-22px
   - Each streak: blend pixel toward 200 brightness at alpha=0.25-0.55
```

Autumn (optional)

```
Simulates: Autumn foliage, warm-brown color shift

1. Brightness x0.95, Saturation x1.1
2. Boost red channel x1.18
3. Reduce green channel x0.88
4. Reduce blue channel x0.75
```

Spring (optional)

```
Simulates: Fresh growth, vibrant greens, soft warm light

1. Brightness x1.15, Saturation x1.3, Contrast x0.95
2. Boost green channel x1.12
3. Slight red boost x1.05 (warm morning light)
```

7.3 Output

Season	Images per species	Total (12 species)
Summer	~25	~300
Winter	~25	~300
Monsoon	~25	~300
Total	~75	~900

8. Final Dataset Summary

Phase	Variant Type	Images per species	Total
Seed collection	Raw (iNat + GBIF)	~28	~336
Lighting GAN	Bright / Dim / Dark	~75	~900
Weather simulation	Summer / Winter / Monsoon	~75	~900
Grand Total		~178	~2,136

Dataset Structure

```
rare_medicinal_plants/
    raw_images/                 <- seed images from iNat + GBIF
        Rauvolfia_serpentina/
        Nardostachys_jatamansi/
        ... (12 species)
    synthetic_lighting/         <- original / bright / dim / dark
        ... (12 species x 4 variants)
    synthetic_weather/          <- summer / winter / monsoon
        ... (12 species x 3 variants)
    rare_medicinal_plants_metadata.csv
    synthetic_metadata.csv
    synthetic_weather_metadata.csv
```

9. Tools & Environment

Tool	Version	Purpose
Python	3.13	Core language
PyTorch	2.x	GAN training
torchvision	latest	Image transforms
OpenCV (cv2)	4.x	GrabCut segmentation
Pillow (PIL)	<13.0	Image I/O, photometric transforms
NumPy	1.x	Array operations
Pandas	2.x	Metadata CSV management
Matplotlib	3.x	Visualization
Google Colab	T4 GPU	Training platform
iNaturalist API	v1	Seed data - field photos
GBIF API	v1	Seed data - herbarium specimens

Known Technical Constraints

Issue	Root Cause	Resolution
‘rembg‘ ImportError	NumPy conflict in Colab Python 3.13	Replaced with OpenCV GrabCut
‘inplace=True‘ autograd error	Skip connections in ResNet/U-Net	All activations set ‘inplace=False‘
GrabCut background bleeding	Similar plant/background color	Identified as limitation; SAM recommended for
LSGAN vs BCE	BCE vanishing gradients	Switched to MSE (LSGAN) for CycleGAN
DiffAugment IndexError	PyTorch 4D tensor slice bug	BHWC permute -> index -> BCHW

Documentation generated: November 2026

Project: Synthetic Dataset Generation for Rare Medicinal Plant Classification